

CMPSC 473 PROJECT 1

Systems Programming, Address Spaces, and Processes

Project Report

This report contains the predictions, explanations, debugging evidence, address-space observations, memory-error diagnoses, and process observations completed during the process of completing Project 1. Numeric memory addresses shown below are from my recorded runs and can vary between executions.

Part I - Pointers and Memory Refresher

1. Types and Referenced Objects

p has type `int *` and points to `a[0]`. pp has type `int **` and points to the pointer variable p. *pp has type `int *` and points to `a[0]`. **pp has type `int` and represents the integer stored in `a[0]`.

2. Difference Between p, &p, *p, a, and &a

p contains the address of `a[0]`. &p is the address of the pointer variable p itself. *p is the integer stored at the address contained in p. In most expressions, a becomes a pointer to its first element. &a points to the entire array. It has a different type from a, even though they normally have the same starting numeric address.

3. Meaning of p + 1

Pointer arithmetic moves according to the size of the pointed-to object. Because p is an `int *`, p + 1 points to the next integer in memory. If p has address A, then p + 1 is normally: A + sizeof(int). On this system, an integer is typically 4 bytes.

4. First printf Prediction

After `update(&a[1], heap)`, `a[1]` changes from 20 to 30 and `heap->y` becomes 30. The first call therefore prints 10 30 30.

5. Storage Locations

The variables p, pp, a, local, and the pointer variable heap are local variables stored on the stack. The structure that heap points to was dynamically allocated and is stored on the heap.

6. Size of Explanation

`malloc(sizeof *heap)` allocates enough bytes for one object of the pointed-to type without hard-coding a size. `sizeof(heap)` measures the pointer itself, whereas `sizeof(*heap)` measures the pointed-to struct pair.

7. Dynamic Array Modification

```
struct pair *objects = malloc(100 * sizeof *objects);
if (objects == NULL)
    return 1;
for (size_t i = 0; i < 100; ++i) {
    objects[i].x = (int)i;
    objects[i].y = 2 * (int)i;
}
free(objects);
```

Part II - Make and Separate Compilation

The completed Makefile builds an executable named stats. It compiles main.c and stats.c separately into object files and includes the dependency on stats.h. The compiler flags include -Wall, -Wextra, and -g. A clean target is also provided.

Rebuild Experiment:

Running make again immediately after a successful build rebuilt nothing because all targets were up to date. After modifying only stats.c, Make recompiled stats.o and relinked stats, while main.o did not need recompilation. After touching/modifying stats.h, the dependent object files were rebuilt and the executable was relinked. Compiling produces object files from source; linking combines object files into the final executable.

Part III - Debugging with GDB

Observed failure and investigation

The original program could print total = 100, but GDB exposed an invalid loop iteration. A breakpoint in total_scores showed n = 4 and valid records {100,10}, {101,20}, {102,30}, and {103,40}. Stepping showed i progressing through 0, 1, 2, 3 and then reaching 4.

At i = 4, records[i].score accesses records[4], one element past the four-element allocation. A backtrace showed the access in total_scores, called from main. This is invalid C memory behavior even if the program happens to print the expected total.

Fix

```
for (int i = 0; i < n; i++) {
    total += records[i].score;
}
```

Changing i <= n to i < n limits access to valid indices 0-3. After rebuilding, the corrected program printed total = 100

Part IV - Virtual Address Spaces and /proc

1. Stack Versus Heap Addresses

The values printed using `& local` are addresses of local variables created during recursive function calls. These variables are stored on the stack. The values printed as heap are addresses returned by dynamic memory allocation and are located on the heap. The addresses were compared against `/proc/PID/maps`, which confirmed that the local-variable addresses were inside the `[stack]` region and the dynamically allocated addresses were inside the `[heap]` region.

2. Address Direction During Recursion

In the recorded run, stack addresses decreased as recursion deepened, from about `0x7ffc3cbe7c` at depth 0 toward `0x7ffc3cbbac` at depth 9. Heap allocations increased in this run, from about `0x5b9ff83372a0` toward `0x5b9ff83377d0`. These are observations from this run, not guarantees made by C.

3. /proc/PID/maps

For PID 14702, the important mappings included:

```
[heap] 5b9ff8337000-5b9ff8358000 rw-p
[stack] 7ffc3ad000-7ffc3ce000 rw-p
```

The printed heap addresses were located inside the `[heap]` range. The printed local-variable addresses were located inside the `[stack]` range.

4. Virtual Memory Versus Physical Memory

For PID 14702, `VmSize` was 2684 kB and `VmRSS` was 1408 kB. Each observed heap/stack mapping spans 0x21000 bytes, about 132 KiB. Virtual mappings describe address ranges available to the process, whereas RSS describes pages currently resident in physical memory; mapped virtual size therefore differs from actual RAM residency.

5. strace System Calls

Several important system calls appeared in the trace.

- `execve` started the executable and established the program image.
- `brk` was related to management of the process heap.
- `mmap` created virtual-memory mappings used by the program and runtime libraries.
- `write` was used when the program printed its depth, addresses, and prompts.
- `read` was used when the program waited for ENTER from standard input.

Part V - Memory Errors

Call	Error Category	Offending Statement	One-Sentence Explanation	Fix
Test 1	Out-of-bounds write	<code>for (size_t i = 0; i <= n; ++i)</code>	The <code><= n</code> condition allows the loop to access <code>a[n]</code> , which is one element beyond the allocated array.	Change the condition to <code>i < n</code> so only valid array indexes are accessed.
Test 2	Use of uninitialized memory	<code>initialize_prefix(a, n - 1);</code>	Only the first <code>n - 1</code> elements are initialized, but <code>a[n - 1]</code> is later read by <code>printf()</code> .	Change the call to <code>initialize_prefix(a, n);</code> so every element is initialized before use.
Test 3	Use after free	<code>printf("test3: %d\n", *p);</code> after <code>release_int(p);</code>	The program dereferences <code>p</code> after the memory it points to has already been freed.	Move <code>printf()</code> before <code>release_int(p);</code> so the value is read while the allocation is still valid.
Test 4	Memory leak	<code>char *message = make_message();</code> without a later <code>free(message);</code>	<code>make_message()</code> dynamically allocates 64 bytes that are never released before the function returns.	Add <code>free(message);</code> after the message is printed.

Part VI - Processes, fork, and exec

1. Parent/child PIDs and addresses after fork

The parent PID was 14906 and the child PID was 14907. Immediately after `fork`, both printed global `0x5f1bd73e6010`, stack `0x7ffcdffe6e4`, and heap `0x5f1c0e8c22a0`. These equal numeric values are virtual addresses in separate process address spaces and do not mean the ordinary objects are one shared physical-memory location.

2. Child writes versus parent values

The child changed its values from 100,200,300 to 101,201,301. After the child finished, the parent still printed 100,200,300. Thus ordinary writes in the child do not alter the parent's corresponding variables; `fork` creates separate process address spaces, commonly implemented using copy-on-write.

3. /proc/PID/maps before and after exec

Before exec, child PID 14907 mapped the processes/parent executable. After exec, the same PID mapped processes/child, and its executable, heap, and stack ranges changed. The new program printed global 0x5757b7082010, stack 0x7ffc37f8dbdc, heap 0x5757f646e2a0, and values 900,901,902. Successful exec therefore preserves the process identity/PID while replacing the program image and address-space mappings.

4. strace -f evidence

```
14943 clone(..., SIGCHLD, ...) = 14944
```

```
14944 execve("./child", ["child"], ...) = 0
```

```
14943 wait4(14944, ..., 0, NULL) = 14944
```

clone is the fork/clone-related process-creation call observed; execve replaced the child's program image; and wait4 was the parent's wait-related system call.